Synthesis of Multimodal Video Processing: Architectures for Automated Transcription, Semantic Summarization, and Intelligent Visual Documentation

The paradigm of knowledge management has undergone a fundamental shift with the ubiquity of high-definition screen recording and video conferencing. While the ability to capture digital interactions is now a standard feature across operating systems, the subsequent transformation of these raw temporal assets into structured, searchable, and semantically rich documentation remains a complex multidisciplinary challenge. The user's requirement for a system that not only transcribes and summarizes but also "intelligently" extracts only the most relevant visual snapshots reflects a sophisticated need for context-aware media processing. This report examines the current state of open-source technologies capable of facilitating this pipeline, ranging from audio-visual normalization and high-fidelity transcription to advanced keyframe extraction algorithms that leverage causal graphs and multimodal embeddings.

Foundations of Automated Video Ingestion and Media Normalization

A robust video processing pipeline begins with the ingestion and normalization of media files. Whether the source is a live screen capture or a pre-recorded MP4, the system must ensure that the audio and video streams are in a format compatible with downstream artificial intelligence models. Open-source frameworks such as FFmpeg and yt-dlp provide the necessary infrastructure for these preliminary stages.[1] Normalization is not merely a matter of file extension; it involves the standardization of codecs, sample rates, and bitrates to ensure predictable behavior in the transcription and computer vision layers.

Standardizing Media Containers and Codecs

The normalization to an MP4 container, specifically utilizing the H.264 or H.265 video codec and AAC or PCM audio, is essential for cross-platform compatibility. High-fidelity transcription models, such as OpenAI Whisper, are particularly sensitive to audio quality. Research suggests that extracting audio as a 16kHz mono WAV file (PCM 16-bit) provides the optimal balance between accuracy and processing speed.[3] In mono recording, the complexity of the signal is reduced, allowing the acoustic encoder to focus on speech features without the interference of spatial data.

Normalization Parameter	Targeted Configuration	Rationale
Video Container	MP4 (MPEG-4 Part 14)	Universal support across web and desktop players.[1]
Video Codec	H.264 (libx264)	Balance of compression efficiency and decoding speed.[4]
Audio Sample Rate	16,000 Hz (16kHz)	Native sampling rate for many STT models, including Whisper.[3]
Audio Channels	Mono (1 Channel)	Simplifies speech-to-text processing and reduces file size.[3]
Bit Depth	16-bit PCM	Ensures sufficient dynamic range for clear transcription.[3]

Event-Driven vs. Polling-Based Screen Capture

A significant distinction in open-source screen recording lies in the capture mechanism itself. Traditional recorders use a polling-based approach, capturing frames at a fixed rate (e.g., 30 frames per second). This often results in thousands of redundant images, especially during presentations or programming sessions where the screen remains static for long periods.

Advanced open-source solutions like Screenpipe introduce an event-driven capture model. Instead of recording every second, the system listens for OS-level triggers such as application switches, window changes, mouse clicks, or typing pauses.[5] This ensures that the system only captures a screenshot when something "meaningful" happens. This approach directly addresses the user's requirement to avoid saving still images of a desktop that are not moving or adding value. By pairing the screenshot with accessibility tree data—the structured text the OS already possesses about buttons and labels—the system achieves high data density with minimal CPU overhead.[7]

The Audio-Transcript Backbone: High-Fidelity Speech Recognition

The core semantic layer of the proposed app is the transcript. Converting spoken dialogue into text serves as the primary anchor for summarization, chaptering, and, crucially, visual relevance

matching.

Leveraging the Whisper Transformer Architecture

The OpenAI Whisper model has become the de facto standard for open-source speech-to-text. It is a sequence-to-sequence Transformer model trained on a vast multilingual dataset, allowing it to handle diverse accents and technical jargon with high accuracy.[8] Implementations such as Faster-Whisper use C++ backends and quantization to provide transcription speeds that are significantly faster than real-time.[1]

For the user's specific need to identify "worthy" images based on what is being discussed, word-level timestamps are indispensable. Whisper achieves this by analyzing the cross-attention patterns between the audio encoder and the text decoder, using Dynamic Time Warping (DTW) to align specific words with their precise moment in the audio stream.[9]

Model Variant	Memory Usage (Approx.)	Accuracy Level	Recommended Use Case
Whisper Tiny	150MB	Low-Medium	Real-time monitoring, low-power devices.[1]
Whisper Base	250MB	Medium	Fast transcription with decent accuracy.[1]
Whisper Small	750MB	Medium-High	Balanced performance for general meetings.[1]
Whisper Medium	1.5GB	High	Technical lectures with specialized vocabulary.[1]
Whisper Large-v3	3GB+	State-of-the-Art	Maximum accuracy for complex documents.[1]

Speaker Diarization and Audio Splitting

In environments with multiple participants, such as team standups or interviews, speaker identification (diarization) adds a critical layer of context to the documentation. Open-source

libraries like pyannote.audio can be integrated into the Whisper pipeline to label speaker turns (e.g., "Speaker 1," "Speaker 2").[6] Furthermore, for extremely long videos, the transcription job may be split into 5-minute segments to avoid memory overflow and allow for parallel processing, with the resulting SRT or VTT files merged during the final synthesis.[11]

Intelligent Keyframe Extraction: The Semantic-Visual Link

The most complex requirement specified is the "intelligent processing" of visual snapshots. The system must move beyond simple scene detection to understand which frames are "worth having" based on the spoken context.

Traditional Scene Detection and Its Limitations

Traditional Scene Change Detection (SCD) calculates the pixel-wise difference between frames. If the difference exceeds a certain threshold (often default to 0.1-1.0 in FFmpeg), a scene break is recorded.[12] While effective for cinema, SCD often fails in screen recordings. A mouse cursor moving across a static slide might trigger a false positive, while a subtle but important text change on a slide might be ignored.

To combat this, a two-stage detector is often employed. The first stage uses Optical Character Recognition (OCR) to detect slide boundaries based on text changes. If the OCR output remains stable, the frame is kept; if it changes significantly, a new boundary is marked.[14] The second stage acts as a visual fallback, using algorithms like Structural Similarity Index (SSIM) to detect changes that OCR might miss, such as a new diagram or image.[14]

Contextual Relevance Matching via CLIP and LLM Reasoning

The true "intelligence" of the system is derived from Vision-Language Models (VLMs) like CLIP (Contrastive Language-Image Pre-Training). CLIP projects both images and text into a shared high-dimensional embedding space, allowing the system to calculate a "relevance score" between a spoken sentence and a video frame.[15]

The workflow for this "intelligent snapshotting" follows a sophisticated pattern:

1. **Candidate Selection**: The system extracts frames at scene changes or fixed intervals (e.g., every 5-10 seconds).[16]
2. **Semantic Keyword Extraction**: An LLM analyzes a segment of the transcript and identifies key nouns, technical terms, or visual descriptions.[14]
3. **Embedding Comparison**: The system generates CLIP embeddings for both the candidate frames and the keywords. It then calculates the cosine similarity:

$$\text{similarity} = \frac{\mathbf{v} \cdot \mathbf{t}}{||\mathbf{v}||\,||\mathbf{t}||}$$

where $\mathbf{v}$ is the visual embedding and $\mathbf{t}$ is the textual embedding.[15]

4. **Threshold Filtering**: Only frames with a similarity score above a specific threshold (e.g., 0.35) are considered "worthy" of inclusion in the final document.[15]

This ensures that if a speaker says, "As you can see in this architectural diagram," the system identifies the exact frame containing a diagram and includes it, while ignoring the 30 seconds of static desktop that preceded it.[17]

Advanced Algorithms: ILKE-TCG and Tactical Reasoning

For scenarios requiring even higher precision, researchers have developed algorithms like ILKE-TCG (Intelligent Keyframe Hierarchical Extraction - Temporal Causal Graph). This approach fusses image structure, voice rhythms, and action paths to construct a heterogeneous temporal graph.[19] By using a graph attention mechanism and Laplace spectral space mapping, the system can estimate cross-modal node weights and deconstruct causal signals.[19] This level of processing allows the AI to understand not just that a frame has changed, but *why* it is semantically important in a "threat causal chain" or a complex technical sequence.[20]

Large Multimodal Models (LMMs) for Video Comprehension

The evolution from simple embeddings to Large Multimodal Models like LLaVA (Large Language and Visual Assistant) provides a new frontier for automated documentation. These models can "see" the video content and reason about it in natural language.

Zero-Shot Modality Transfer with LLaVA-NeXT

LLaVA-NeXT utilizes a technique called "AnyRes," which allows the model to digest high-resolution images by segmenting them into grids (e.g., 2x2 or 1xN). This technique naturally generalizes to video by treating multiple frames as a sequence of visual tokens.[21] Remarkably, LLaVA-NeXT demonstrates strong zero-shot performance in video understanding even when trained primarily on image-text pairs.[21] This means the model can identify objects, read stylized text that traditional OCR might struggle with, and understand context-dependent visual signals.[16]

The SlowFast Representation for Dense Frame Sampling

To overcome the GPU memory constraints inherent in processing long videos, LLaVA-Video introduces a "SlowFast" representation.[22] This method optimally distributes visual tokens across different frames, allowing the model to incorporate up to three times more frames than traditional methods. This ensures that the AI doesn't miss rapid visual transitions or brief but critical on-screen messages, fulfilling the user's desire for a system that "analyzes all screens"

but only extracts the "useful ones".[22]

Feature	CLIP-based Matching	LMM (LLaVA-Video) Reasoning
Logic Type	Statistical similarity (embeddings)	Natural language reasoning and context.[23]
Speed	Extremely fast, suitable for real-time.[15]	Slower, typically asynchronous.[3]
Contextual Depth	Limited to keyword/frame similarity.	Understands "why" a frame is important.[16]
Text Handling	Struggles with stylized or obfuscated text.	High-resolution OCR and context discovery.[16]
Primary Tool	OpenCV + Sentence-Transformers.[3]	PyTorch + LLaVA weights.[24]

Automated Document Synthesis: Creating the Final Asset

The final requirement involves synthesizing the summary, transcript, and intelligently selected images into a professional document. This requires a programmatic approach to document generation.

Programmatic Formatting with Spire.Doc and MarkItDown

Open-source and professional-grade Python libraries allow for the automated creation of Microsoft Word (.docx) and PDF files. Microsoft's markitdown utility is specifically designed to convert office documents and images into Markdown, preserving structure like headings and tables for use with LLMs.[25] For the reverse process—generating a Word doc from AI output—Spire.Doc for Python provides granular control over the document object model (DOM).[26]

A typical synthesis workflow follows these steps:

1. **Markdown Generation**: The system creates a Markdown file containing the AI summary, chapter headers, and transcript segments.
2. **Image Insertion**: At the precise timestamps identified by the CLIP/LMM layer, the system inserts Markdown image links (e.g., !(frame_123.jpg)).[12]

3. **High-Fidelity Conversion**: Using a library like Pandoc or Spire.Doc, the Markdown is converted to a .docx file. During this process, specific formatting rules are applied—for example, ensuring images are centered and titles use Heading styles.[28]

Handling Dynamic Data and Templates

For corporate or academic use, the system can use pre-defined templates. The Spire.Doc library supports placeholder replacement, allowing the app to inject the video's title, duration, and AI-generated summary into a formatted report template with company branding.[26] This ensures that the output is not just a text file, but a professional-grade document ready for distribution.

Review of Leading Open-Source Projects and Frameworks

To implement the desired functionality, several open-source projects provide ready-made components or entire pipelines that can be adapted.

1. summarize (steipete/summarize)

This is a highly versatile tool that can point at any URL, YouTube video, or local file and generate a summary. It includes a Chrome Side Panel and a CLI.[12]

- **Visual Capability**: It features "Video slides," which include screenshots, OCR text, and transcript cards. It uses yt-dlp for downloads, FFmpeg for frame extraction, and Tesseract for OCR.[12]
- **Mechanism**: It identifies "media-friendly" URLs and triggers an extraction pipeline that runs in parallel with the summary generation. It can render these slides directly in a browser sidebar or an iTerm2-supported terminal.[12]

2. screenpipe (screenpipe/screenpipe)

screenpipe is perhaps the closest solution for the recording portion of the user's request. It continuously captures the screen and audio 24/7 and stores the data locally.[5]

- **Visual Capability**: It uses event-driven capture to avoid redundant screenshots. It provides an "automation store" where users can install "Pipes"—AI agents that perform specific tasks.[2]
- **Specific Pipe**: The meeting-summary pipe auto-transcribes and summarizes sessions, while the obsidian-sync pipe can sync activity as daily markdown notes with connections.[2]

3. video-transcriber (Romilly Cocking)

This specific project focuses on the exact output format requested: a portable ZIP file

containing a Markdown transcript and slide images.[27]

- **Visual Capability**: It uses "Smart slide detection" based on perceptual hashing to capture only distinct frames.[27]
- **Mechanism**: It uses Whisper AI locally for transcription and performs "Timeline merging" to associate the audio with the corresponding slides, creating a clean, structured documentation asset.[27]

4. AI-Video-Transcriber (wendy7756)

A production-ready web application for transcribing and summarizing videos and podcasts.[1]

- **Stack**: Built with FastAPI, yt-dlp, Faster-Whisper, and the OpenAI API.[1]
- **Visual Capability**: While focused primarily on text, its architecture provides the necessary hooks for integrating frame extraction, as it already handles the video downloading and audio isolation stages.[1]

5. PPTAgent and Paper2Video

These specialized tools focus on the relationship between presentations and videos. PPTAgent analyzes reference presentations to extract slide-level functional types, while Paper2Video integrates slide generation with speech synthesis and cursor grounding.[30] These represent the cutting edge of "document-to-video" and "video-to-document" translation, utilizing "Tree Search Visual Choice" for refined layouts.[32]

Architecture and Deployment Considerations

Designing an app to perform these tasks requires a balance between local privacy and cloud-based intelligence. The user's request for "free open source" suggests a preference for local models where possible.

The Hybrid Cloud-Local Pipeline

To optimize for cost and speed, a hybrid architecture is recommended.

- **Local Processing**: Audio normalization, frame extraction (FFmpeg), and initial scene detection should happen locally to avoid massive data uploads.[3]
- **Edge AI**: Transcription (Whisper) can be run locally on a mid-range GPU (8GB+ VRAM), ensuring that sensitive meeting audio never leaves the machine.[1]
- **Cloud reasoning**: For complex summarization or visual relevance matching using the highest-tier models (like GPT-4o or Gemini 1.5 Pro), the system can send small metadata packets—text segments and low-resolution frame samples—to a cloud API.[18]

Hardware and Resource Requirements

Processing video is resource-intensive. A typical deployment requires significant RAM and GPU

acceleration to be viable for professional use.

Requirement	Minimum (Standard)	Recommended (Professional)
RAM	4GB - 8GB.[1]	16GB - 32GB (for large Whisper models).[1]
CPU	Dual-core.[1]	Quad-core or higher (for FFmpeg).[1]
GPU	Not required (CPU only).	NVIDIA A6000 or RTX 4090 (48GB VRAM for LMMs).[31]
Disk Space	2GB for app installation.	SSD with 20GB+ for monthly data storage.[1]
Processing Speed	0.5x - 1x real-time (slow).	2x - 5x real-time (fast).[6]

Docker and Server Deployment

For a server-side implementation of the "upload and process" functionality, Docker is the preferred deployment method. A Docker container ensures that dependencies like FFmpeg and the specific Python environment are isolated.[1]

- **Memory Overhead**: An idle container requires approximately 128MB, but this can spike to 2GB or more during Whisper processing.[1]
- **GPU Integration**: The nvidia-container-toolkit must be installed on the host to allow the Docker container to access GPU resources for transcription and CLIP embedding generation.[34]

Technical Roadmap for Integration into a Custom App

For a developer looking to integrate these features, the following technical roadmap provides a structured path from ingestion to documentation.

Stage 1: The Ingestion Engine

Implement a watch folder or upload endpoint using FastAPI. As soon as an MP4 is detected, trigger an FFmpeg worker to extract the audio stream as a 16kHz WAV and the video frames as a low-FPS sequence (e.g., 1 frame every 5 seconds) into a temporary workspace.[3]

Stage 2: The Semantic Encoder

Run faster-whisper on the WAV file. Ensure the --word_timestamps True flag is set to get a JSON output containing the exact start and end times for every word.[9] Concurrently, run an LLM to generate a summary and identify "semantic anchors"—points in the transcript where visual evidence is required (e.g., "Look at this table...").[12]

Stage 3: The Visual Filter

Perform perceptual hashing on the extracted frames to remove those that are visually identical (addressing the "static desktop" problem).[27] For the remaining frames, calculate CLIP embeddings. Compare these embeddings against the "semantic anchors" identified in Stage 2.[15] Select the frame with the highest similarity score for each anchor.

Stage 4: The Document Architect

Use a library like Spire.Doc or Pandoc. Create a document structure:

- **Header**: Title and Date.
- **Executive Summary**: AI-generated gist.
- **Content Sections**: For each chapter, include the relevant text followed by the intelligently selected snapshot.[12]
- **Footer**: Full transcript (optional) or key action items.

Stage 5: Optimization and Cleanup

Once the document is generated, the system should purge the temporary WAV file and the thousands of discarded frames to save storage space.[3] The final result is a single.docx or.pdf file that captures the "essence" of the video without the "noise" of redundant visual data.

Challenges in Multi-Modal Temporal Alignment

While the technology for these tasks exists, achieving perfect "alignment" remains a challenge. A speaker might refer to a slide five seconds before it appears or ten seconds after it has been changed.

To solve this, researchers utilize "Context-aware Video-text Alignment" (CVA) frameworks.[36] These systems use a "replacement clip" strategy to simulate diverse contexts and a "Context-enhanced Transformer Encoder" to capture hierarchical temporal relationships.[36] In simpler terms, the system looks at a 15-second "window" around the timestamp to find the best possible visual match, rather than just the exact millisecond the word was spoken. This "fuzzy matching" significantly improves the perceived intelligence of the automated documentation.[18]

Another approach involves "Iterative Refinement".[38] The system first selects a median frame from a segment and generates a caption. It then compares the caption to the transcript. If the alignment is poor, it predicts a better keyframe and regenerates the caption. After four cycles of this iterative process, caption scores (like BLEURT) and keyframe matching scores

significantly improve, leading to more accurate summaries for the human reader.[38]

Nuanced Conclusions and Actionable Recommendations

The demand for a free, open-source application that can "intelligently" document screen activities is met by a combination of existing tools and advanced algorithms. Based on the analysis of current technologies, the following strategies are most effective:

- **For Immediate Use**: Utilize Screenpipe for the recording and initial summarization stages. Its "Pipes" architecture allows for the direct creation of meeting notes and Obsidian-compatible documents with embedded images.[2]
- **For Custom Application Development**: Build a pipeline using FFmpeg for normalization, faster-whisper for word-level timestamps, and CLIP for semantic-visual relevance matching. This combination provides the most precise control over which images are deemed "useful".[3]
- **To Solve the "Static Desktop" Problem**: Implement perceptual hashing (pHash) during the frame extraction phase. This is a low-compute way to discard non-informative frames before they even reach the AI models, saving significant processing time.[27]
- **For High-Resolution Documentation**: Integrate LMMs like LLaVA-NeXT for the final reasoning stage. Their ability to read stylized text and understand complex UI context makes them superior to traditional OCR-based systems for generating professional-grade summaries of technical videos.[16]
- **For Scalability**: Deploy the entire stack in a Dockerized environment with GPU passthrough. This allows for the parallel processing of multiple videos and ensures consistent behavior across different hardware setups.[1]

In summary, the transition from raw video to "intelligent" documentation is no longer a futuristic concept but a deployable reality. By leveraging the cross-modal alignment capabilities of modern Transformers and the efficient capture mechanisms of event-driven screen recording, developers can create tools that truly understand the digital work being captured, presenting only the most vital visual and textual information to the user.

Works cited

1. Transcribe and summarize videos and podcasts using AI. Open-source, multi-platform, and supports multiple languages. - GitHub, accessed April 14, 2026, https://github.com/wendy7756/AI-Video-Transcriber
2. Screenpipe — Screen AI That Records Everything & Automates Anything, accessed April 14, 2026, https://screenpi.pe/
3. Beyond Keywords: How I Built an AI-Powered Video Search System with OpenAI Whisper & FAISS | by Samad Khan | Medium, accessed April 14, 2026, https://medium.com/@samadk619/beyond-keywords-how-i-built-an-ai-powered

-video-search-system-with-openai-whisper-faiss-e9dcc41ac79b
4. How to Extract Frames from Video in Python - Cloudinary, accessed April 14, 2026, https://cloudinary.com/guides/video-effects/how-to-extract-frames-from-video-in-python
5. What is screenpipe? Open source AI screen memory, accessed April 14, 2026, https://screenpi.pe/about
6. Architecture - Screenpipe - Mintlify, accessed April 14, 2026, https://mintlify.com/screenpipe/screenpipe/architecture
7. GitHub - screenpipe/screenpipe: Run agents that work for you based on what you do. AI finally knows what you are doing, accessed April 14, 2026, https://github.com/screenpipe/screenpipe
8. Using Whisper and BERTopioc to model Kurzgesagt's videos - Maarten Grootendorst, accessed April 14, 2026, https://www.maartengrootendorst.com/blog/whisper/
9. Word-Level Timestamps - Whisper - Mintlify, accessed April 14, 2026, https://www.mintlify.com/openai/whisper/guides/word-timestamps
10. InfantLab/VideoAnnotator: A tool for automatic annotation of ... - GitHub, accessed April 14, 2026, https://github.com/InfantLab/VideoAnnotator
11. How to use whisper to handle long video? - API - OpenAI Developer Community, accessed April 14, 2026, https://community.openai.com/t/how-to-use-whisper-to-handle-long-video/530862
12. steipete/summarize: Point at any URL/YouTube/Podcast or ... - GitHub, accessed April 14, 2026, https://github.com/steipete/summarize
13. How to extract frames from a video? - Shotstack, accessed April 14, 2026, https://shotstack.io/learn/extract-frames-from-video/
14. A Slide Annotation System with Multimodal Analysis for Video Presentation Review - MDPI, accessed April 14, 2026, https://www.mdpi.com/1999-4893/19/2/110
15. Clip - Roboflow Inference, accessed April 14, 2026, https://inference.roboflow.com/foundation/clip/
16. GitHub - sevkaz/Vision_Language_Video_Scanner: VLM-first video frame scanner that analyzes video frames with a vision-language model and optional OCR., accessed April 14, 2026, https://github.com/sevkaz/Vision-Language-Video-Scanner-
17. Using Whisper for audio transcription in documentary footage - YouTube, accessed April 14, 2026, https://www.youtube.com/shorts/bzvtfrM7HfA
18. Context-Aware Video Retrieval and Clipping Using Text Queries - ResearchGate, accessed April 14, 2026, https://www.researchgate.net/publication/390498451_Context-Aware_Video_Retrieval_and_Clipping_Using_Text_Queries
19. A Tactical Behaviour Recognition Framework Based on Causal Multimodal Reasoning: A Study on Covert Audio-Video Analysis Combining GAN Structure Enhancement and Phonetic Accent Modelling - Preprints.org, accessed April 14,

2026, https://www.preprints.org/manuscript/202507.1431
20. A Tactical Behaviour Recognition Framework Based on Causal Multimodal Reasoning: A Study on Covert Audio-Video Analysis Combinin - arXiv, accessed April 14, 2026, https://arxiv.org/pdf/2507.21100
21. LLaVA-NeXT: A Strong Zero-shot Video Understanding Model, accessed April 14, 2026, https://llava-vl.github.io/blog/2024-04-30-llava-next-video/
22. LLaVA-Video: Video Instruction Tuning With Synthetic Data - arXiv, accessed April 14, 2026, https://arxiv.org/html/2410.02713v3
23. LLaVA-Video: Video Instruction Tuning With Synthetic Data | OpenReview, accessed April 14, 2026, https://openreview.net/forum?id=EEIFGvt39K
24. Introducing Video-LLaVA: Unifying Vision and Language AI for Images and Videos | by Tanveer Ahmed Khan | Medium, accessed April 14, 2026, https://medium.com/@takhan.11/introducing-video-llava-unifying-vision-and-language-ai-for-images-and-videos-59fc61fa7233
25. microsoft/markitdown: Python tool for converting files and office documents to Markdown. - GitHub, accessed April 14, 2026, https://github.com/microsoft/markitdown
26. How to Convert Markdown to Word Documents Using Python - DEV Community, accessed April 14, 2026, https://dev.to/allen_yang_f905170c5a197b/how-to-convert-markdown-to-word-documents-using-python-1alj
27. video-transcriber: extract frames from videos and transcribe audio - Adafruit Blog, accessed April 14, 2026, https://blog.adafruit.com/2025/12/10/video-transcriber-extract-frames-from-videos-and-transcribe-audio/
28. Automating Word Document Creation with Python: A Practical Guide - DEV Community, accessed April 14, 2026, https://dev.to/allen_yang_f905170c5a197b/automating-word-document-creation-with-python-a-practical-guide-21lf
29. Pandoc - index, accessed April 14, 2026, https://pandoc.org/
30. GitHub - icip-cas/PPTAgent: An Agentic Framework for Reflective PowerPoint Generation, accessed April 14, 2026, https://github.com/icip-cas/pptagent
31. showlab/Paper2Video: Automatic Video Generation from Scientific Papers - GitHub, accessed April 14, 2026, https://github.com/showlab/Paper2Video
32. Paper2Video: Automatic Video Generation from Scientific Papers - arXiv, accessed April 14, 2026, https://arxiv.org/html/2510.05096v2
33. GitHub - martinopiaggi/summarize: Video transcript summarization from multiple sources (YouTube, Instagram, TikTok, Twitter, Reddit, Facebook, Google Drive, Dropbox, and local files). Works with any OpenAI-compatible LLM provider (even locally hosted)., accessed April 14, 2026, https://github.com/martinopiaggi/summarize
34. Using CLIP Embeddings — Tao Toolkit, accessed April 14, 2026, https://docs.nvidia.com/tao/tao-toolkit/6.26.03/text/multimodal/clip/applications.html
35. Knowledge from Video to text. Automating Video Content Extraction: A... | by

Supriyo Roy Banerjee | Medium, accessed April 14, 2026, https://medium.com/@banerjeesupriyo586/knowledge-from-video-to-text-2abd5da376a2
36. CVA: Context-aware Video-text Alignment for Video Temporal Grounding - arXiv, accessed April 14, 2026, https://arxiv.org/html/2603.24934v1
37. (PDF) ComVi: Context-Aware Optimized Comment Display in Video Playback - ResearchGate, accessed April 14, 2026, https://www.researchgate.net/publication/403263114_ComVi_Context-Aware_Optimized_Comment_Display_in_Video_Playback
38. A Challenging Multimodal Video Summary: Simultaneously Extracting and Generating Keyframe-Caption Pairs from Video | OpenReview, accessed April 14, 2026, https://openreview.net/forum?id=YvzA0hFCF3